

CSCE 620 COMPUTATIONAL GEOMETRY · MODULE 01 · LECTURE NOTES

Geometry a Machine Can Believe

The algorithms are correct. The arithmetic is not.

MODULE OUTCOMES

Explain the structure of a geometric algorithm: predicates plus combinatorics.

Demonstrate a floating-point predicate giving an inconsistent answer.

Explain why inconsistency is worse than inaccuracy.

Name the standard degeneracies and why they are not edge cases.

Set up the tooling this course needs.

1 The structure of a geometric algorithm

EVERY GEOMETRIC ALGORITHM HAS TWO LAYERS

A numerical layer: the predicates. These are small questions with discrete answers — *is this point left of, right of, or exactly on that directed line?, is this point inside, outside, or exactly on that circle?* Each one reduces to the sign of a polynomial in the input coordinates.

A combinatorial layer: the algorithm proper. It builds a structure — a convex hull, a triangulation, a planar subdivision — by asking predicates and branching on their answers, maintaining invariants as it goes.

The correctness proof assumes the predicates are answered exactly. Every proof in every textbook on this subject makes that assumption, and almost none of them state it. The proof is about the combinatorial layer and it takes the numerical layer as given.

Floating point breaks the assumption, and the combinatorial layer has no defence. It cannot detect that it was lied to, because the lie is indistinguishable from a legitimate answer. **That gap is the subject of this course**, and treating it as an implementation detail to be handled later is the single most common way geometric code fails.

$$\text{orient}(a, b, c) = \text{sign}[(b_x - a_x)(c_y - a_y) - (b_y - a_y)(c_x - a_x)]$$

This is twice the signed area of triangle abc : positive when c lies to the left of the directed line $a \rightarrow b$, negative when it lies to the right, and zero when the three are collinear. **Four subtractions, two multiplications, and a final subtraction — seven floating-point operations on which the correctness of convex hulls, triangulations, segment intersection, and point location all rest.** It is worth pausing on how much weight that one expression carries.

2 How it fails

THE FAILURE IS INCONSISTENCY, NOT ERROR

If a predicate is merely *inaccurate*, the result is slightly wrong. That is the familiar situation in numerical computing, it is what error analysis is for, and most engineering tolerates it comfortably.

If a predicate is *inconsistent*, the result is impossible. The algorithm can be told that c is left of ab , that a is left of bc , and that b is left of ca — a configuration that no three points in the real plane can have.

The algorithm's invariants now describe a world that does not exist. A hull-construction loop that pops vertices while they are non-convex may pop the entire stack and index past the bottom. A sweep that assumes an event ordering may process events in an order its state machine has no transition for. **The usual outcomes are an infinite loop, an out-of-bounds access, or a structure that violates its own definition.**

So a geometry bug is rarely 'off by a little'. It is a hang, a crash, or a convex hull that is not convex — and the input that triggers it looks entirely ordinary. **Tolerances do not fix this**, for the reason in §3.

```
double orient(P a, P b, P c) {           // naive, double precision
    return (b.x-a.x)*(c.y-a.y) - (b.y-a.y)*(c.x-a.x);
}

P a{0.5,0.5}, b{12.0,12.0}, c{24.0,24.0};
c.y = nextafter(c.y, 25.0);             // perturb by ONE ulp

orient(a,b,c)  -> +7.1e-15  "left"
orient(b,c,a)  -> -3.6e-15  "right"      <-- contradiction
orient(c,a,b)  ->  0.0      "collinear"  <-- and a third answer

// The same three points, in three cyclic orders that MUST agree,
// return three different answers.
```

Run this before reading further. The cyclic orders of three points describe the same triangle with the same orientation, so all three calls must return the same sign. Watching them disagree is more convincing than any argument about it, and the cyclic-consistency check makes a good permanent test for any predicate implementation.

WHY THE ERROR IS UNBOUNDED, NOT SMALL

The subtractions come first. When the three points are close together relative to their distance from the origin, $b-a$ and $c-a$ lose most of their significant digits to catastrophic cancellation — the leading digits agree and cancel, leaving the trailing rounding noise promoted to the front.

Then the products amplify what remains. The two products are nearly equal and are subtracted from one another, which is a second cancellation applied to values that were already mostly noise.

So the computed result can have no correct digits whatsoever, including its sign. The relative error is not bounded by any small constant; in the worst case it is unbounded, and the computed value bears no relation to the true one.

And the sign is the entire output. A predicate returns one of three discrete values. Losing a few digits of a magnitude is survivable; losing the sign loses the whole answer, and the caller cannot tell that it happened.

3 Why the obvious fixes fail

Attempted fix	Why it does not work
Compare against an epsilon: treat $ \text{orient} < \epsilon$ as collinear.	Makes 'collinear' non-transitive , which breaks sorting and merging — see the callout below. The inconsistency is relocated, not removed, and it moves to inputs that are harder to construct and therefore harder to test.
Use long double, float128, or doubled-double.	Buys digits, not exactness. Every one of these has a finite precision and therefore a cancellation regime. The same failure occurs, on inputs slightly further out, and it is now much rarer — which makes it harder to find and no less fatal.
Snap all inputs to a fixed grid.	A real technique, but it changes the problem , and naive snapping can create degeneracies that were not in the input (two distinct points snapping together) or move a point across a line it was on the other side of. Principled versions exist — snap rounding, Module 13.
Detect failure and retry with a small random perturbation.	Non-deterministic output. The same input produces different results on different runs, which is unacceptable in a build pipeline or an asset tool, and makes bugs unreproducible.
Catch the crash and skip the input.	Converts loud failures into silent wrong answers , which is strictly worse. The crash was the only thing telling you the output was invalid.

EPSILON MAKES EQUALITY NON-TRANSITIVE

With a tolerance, a can be collinear with b , and b collinear with c , while a is not collinear with c . Each pairwise test falls under the threshold; the endpoints together do not.

Algorithms rely on transitivity constantly — to merge runs of collinear points into one edge, to classify a point once and reuse that classification elsewhere, and above all to sort.

A comparator built on an epsilon is not a valid strict weak ordering. This is not a theoretical objection: passing such a comparator to `std::sort` is undefined behaviour, and the standard implementations do in practice run off the end of the sequence and corrupt memory. The crash appears inside the sort, which is nowhere near the geometry that caused it.

So the tolerance does not remove the inconsistency. It moves it from inputs that differ by one ULP — which are at least easy to generate deliberately — to inputs that straddle the threshold, which are not.

Module 02 shows that exactness is affordable, which makes this entire row of trade-offs unnecessary.

4 Degeneracy

- **Three or more collinear points.** Breaks convex hull (which vertices belong?) and triangulation (a zero-area triangle).
- **Four or more cocircular points.** The Delaunay triangulation is then *not unique* (Module 09), and an algorithm that assumes uniqueness has no defined behaviour.

- **Duplicate or coincident points** — and, worse, points that differ by one ULP, which are not equal but are closer together than any predicate can resolve.
- **Segments sharing an endpoint**, collinear overlapping segments, and three or more segments passing through a single point (Module 04).
- **Vertical segments**, whenever the algorithm sweeps or sorts by x and implicitly assumed distinct x-coordinates.
- **Zero-area triangles, zero-length edges, empty or self-intersecting polygons**, and polygons whose first and last vertices coincide.

RANDOM INPUT IS THE UNREPRESENTATIVE CASE

Uniform random points are, with probability one, in general position. No three are collinear, no four cocircular, and no two coincide. This is a theorem, not an observation.

So a naive implementation passes every random test you write, at every size, for as long as you care to run it. Random testing provides essentially no evidence about the robustness of geometric code, which is a genuinely counterintuitive fact given how well it works elsewhere.

And real input is the exact opposite. CAD output, tile grids, terrain heightfields, architectural models, and anything authored by a human are saturated with right angles, shared vertices, axis-aligned edges, and repeated coordinates. **Degeneracy is the common case in real data.**

So test on lattices and grids, on deliberate collinear runs, on duplicated and one-ULP-apart points, on cocircular sets — and on the output of your own earlier passes, which is where the degeneracies you generated yourself come back to be consumed by the next stage.

- **A point and segment type, and an orient behind one interface** whose implementation you can swap. Modules 02 and 13 both replace it.
- **SVG output.** An afternoon's work, and it pays for itself during the first bug. **Geometric errors are obvious when drawn and nearly invisible otherwise** — debugging a triangulation from printed coordinates wastes days.
- **Exact rational arithmetic for ground truth.** Python's `fractions.Fraction` is enough; it is far too slow to ship and perfectly suited to deciding what the right answer was.
- **A degenerate-input generator:** lattice points, collinear runs, duplicates, cocircular sets, near-misses at one ULP, and axis-aligned configurations. **Build it now** — every module after this one uses it, and writing it later means re-testing everything you have already built.
- **The cyclic-consistency check from §2**, as a permanent test applicable to any predicate implementation you write.

Where to see this taught differently

Original material only goes so far — hearing the same idea in another voice is how it sticks. These are free, and they cover this module specifically.

Resource	What it covers for this module
Mount — CMSC 754 lecture notes, Lecture 1 (free PDF)	The framing of §1 and the orientation primitive, done carefully.
Shewchuk — Robust Adaptive Floating-Point Geometric Predicates (free)	The paper behind §2 and §3. Section 1 alone is the best statement of the problem in print, and Module 02 builds its machinery.
Kettner, Mehlhorn, Pion, Schirra & Yap — Classroom Examples of Robustness Problems in Geometric Computations (free)	Worked failures with figures: convex hulls that are not convex, produced by correct algorithms. Short, and it settles the matter for anyone still unconvinced.
Goldberg — What Every Computer Scientist Should Know About Floating-Point Arithmetic (free)	The cancellation analysis of §2, from first principles.

Exercises

Do these in code, not on paper, unless the exercise says otherwise. The module is not finished until they run.

- 1 Implement the naive orient and the cyclic-consistency check. Find an input on which the three cyclic orders disagree.
- 2 **Characterise the failure region:** fix a and b , sweep c over a fine grid near the line, and plot where the computed sign differs from the exact one.
- 3 Repeat the above in float, double, and long double. Report how the failure region shrinks and confirm it does not vanish.
- 4 Implement orient with Fraction and use it as ground truth for the previous two exercises.
- 5 **Build an epsilon-based collinearity test and demonstrate non-transitivity** with three explicit points.
- 6 Write a comparator using that epsilon test and sort with it. Report what happens, and at what input size.
- 7 Implement SVG output for points, segments, and polygons.
- 8 **Write the degenerate-input generator:** lattice points, collinear runs, duplicates, one-ULP pairs, cocircular sets, axis-aligned configurations.
- 9 Generate 10,000 uniform random point sets and verify that none contain three collinear points. Explain the result.
- 10 Take a mesh or CAD file you have and count its degeneracies: duplicate vertices, collinear triples, zero-area faces.

Self-check

Answer these in writing, from memory, without looking back at the notes. If you cannot, you have read this module but not yet learned it — that is useful information, not a failure.

- 1 What are the two layers of a geometric algorithm, and which does the correctness proof concern?
- 2 Write the orientation predicate and say what its sign means.
- 3 Distinguish an inaccurate predicate from an inconsistent one, and say why the second is worse.
- 4 Why can the computed orientation have no correct digits at all?
- 5 Give five attempted fixes and say why each fails.
- 6 Why does an epsilon tolerance break `std::sort`?
- 7 Name six standard degeneracies.
- 8 Why does passing random tests establish nothing about robustness?